Lesson 2 of 5

Keep moving and hit the wall

The square moves on its own and resets at the edge.

Goal	The square moves by itself, and the game restarts if it leaves the screen.
Concepts	State (a variable that remembers), if conditions, abs(), a repeating timer.
Starting point	Replace Lesson 1's code with the code below.
Teacher prep	Draw the diagram on the board: keys -> direction -> loop moves the head.

Session plan (30 minutes)

Time	Activity
0-5	Run last lesson's game. Ask: what is missing? (It only moves when you press a key.)
5-15	Live-code: add head.direction, change the key functions, write move_head() and game_loop().
15-25	Students type and test. They should see the square glide and reset at the edge.
25-30	Speed experiment from the challenge, then wrap-up.

The big idea

Keys no longer move the square. They only **change a variable** (direction). The **game loop** reads that variable and moves the head. Separating input from action is how almost every game works.

The code

```
import turtle

STEP = 20
LIMIT = 280

screen = turtle.Screen()
screen.title("Snake")
screen.bgcolor("black")
screen.setup(600, 600)
screen.tracer(0)          # we'll redraw manually

head = turtle.Turtle("square")
head.color("lime")
head.penup()
head.direction = "stop"   # the head remembers which way it's going

def go_up():    head.direction = "up"
def go_down():  head.direction = "down"
def go_left():  head.direction = "left"
def go_right(): head.direction = "right"

def move_head():
    if head.direction == "up":    head.sety(head.ycor() + STEP)
    if head.direction == "down":  head.sety(head.ycor() - STEP)
    if head.direction == "left":  head.setx(head.xcor() - STEP)
    if head.direction == "right": head.setx(head.xcor() + STEP)

def game_loop():
    if abs(head.xcor()) > LIMIT or abs(head.ycor()) > LIMIT:
```

```
        head.goto(0, 0)
        head.direction = "stop"

    move_head()
    screen.update()
    screen.ontimer(game_loop, 100)   # run again in 100 ms

screen.listen()
screen.onkeypress(go_up, "Up")
screen.onkeypress(go_down, "Down")
screen.onkeypress(go_left, "Left")
screen.onkeypress(go_right, "Right")

game_loop()
screen.mainloop()
```

Key lines explained

- **screen.tracer(0)** turns off automatic drawing. We call **screen.update()** ourselves once per frame, which is smoother.
- **head.direction = "stop"** gives the head a memory. It sits still until a key is pressed.
- **abs()** gives the distance from zero, so -300 and 300 both count as "far". Use a number line to show this.
- **screen.ontimer(game_loop, 100)** runs game_loop again after 100 milliseconds. That is ten frames per second.

Challenge

- Change 100 to 50, then to 200. What does it do to the difficulty?
- Change LIMIT to 200. What changes on screen?
- Stretch: make the head turn red when it resets.

Common problems

What you see	Likely cause and fix
Square does not move at first	That is correct: direction starts as "stop". Press an arrow key.
Nothing is ever drawn or the window looks frozen	screen.update() is missing from game_loop (needed because of tracer(0)).
Square moves once then stops	screen.ontimer(...) is missing, or game_loop() is not called before mainloop().
Game too fast or too slow	Change the 100 in ontimer.

What we learned (student checklist)

[ ] I can store information in a variable that a turtle remembers.

[ ] I can explain why keys change direction instead of moving the head.

[ ] I can use if to check a condition.

[ ] I can make a function run again and again with ontimer.